# HIRESIGHT – JOB SCAM DETECTION PLATFORM

**Name of the project member:**

Sunchu Thrishla Patel(2303A51165)
Batch-18

# INTRODUCTION

**HireSight** is an AI-powered web application designed to detect **fraudulent job postings** by analyzing job descriptions, company details, and suspicious patterns using machine learning techniques.

Many job seekers face scams such as:

- Asking for registration fees
- Fake company offers
- Unrealistic salaries

- This system analyzes job descriptions and warns users about potential scams.

# OBJECTIVES

- Detect fake job postings
- Protect users from fraud
- Provide scam risk score
- Verify company authenticity
- Allow users to report scams

# STAKEHOLDERS

- Job Seekers
- Fresh Graduates
- Companies
- Recruitment Platforms
- Government / Cybercrime Departments

# TECHNOLOGIES USED:

**Programming Languages:**

- HTML
- CSS
- JavaScript

**Backend:**

- Node.js
- Express.js

**Tools & Software:**

- Visual Studio Code
- Browser (Chrome)

# PROMPTS USED

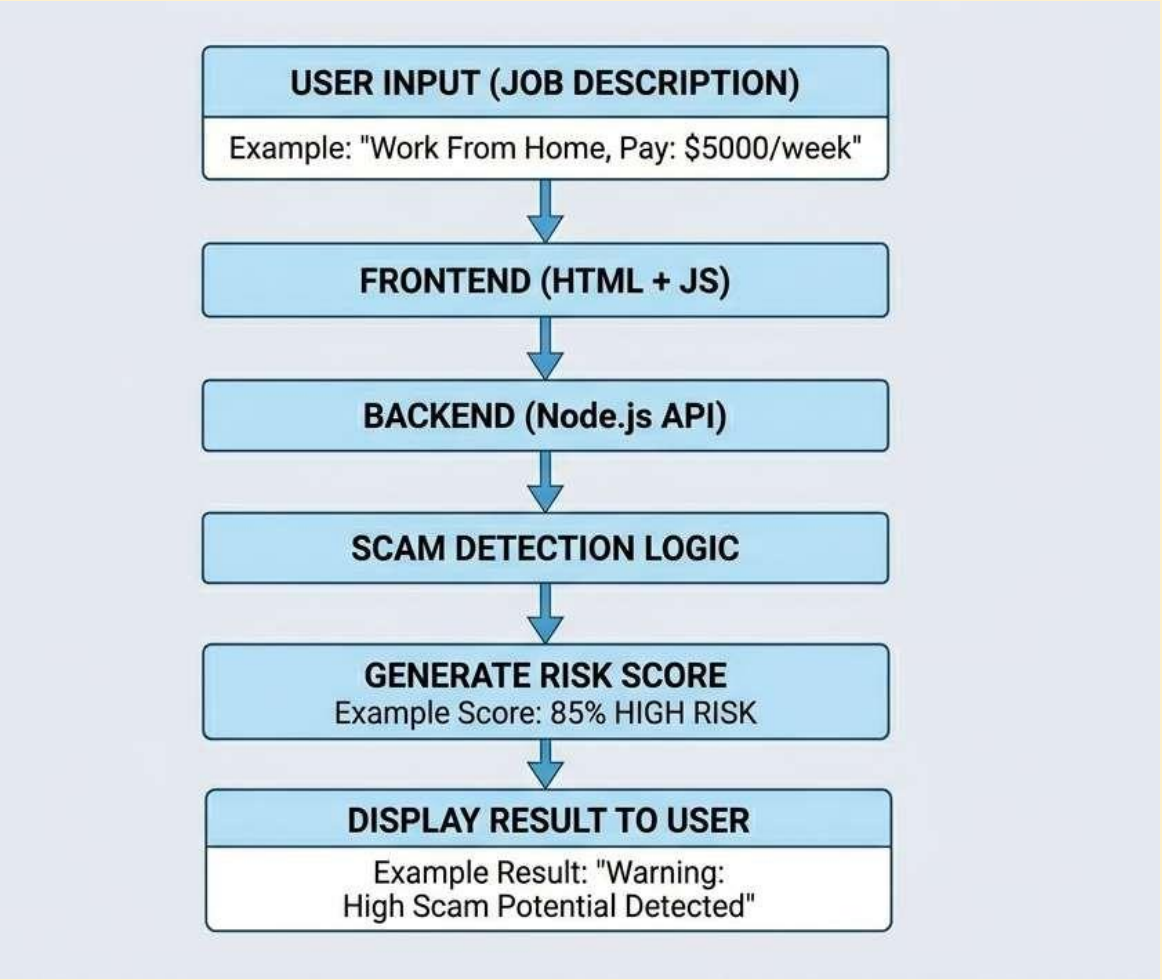
## Prompt Used for Frontend Development

- <!--Generate a complete HTML file for a job scam detection web application named HireSight with clean CSS styling and a responsive layout. Include three sections: Job Scam Checker (textarea + analyze button), Company Verification (input + button), and Report a Scam (multiple inputs + submit button). Use JavaScript with fetch API to connect to a backend at http://localhost:5000 with endpoints /analyze, /company/:name, and /report. Display risk score, risk level with colors (high, medium, low), reasons in a list, company status, and report submission response. Include proper error handling and dynamic result display sections.-->

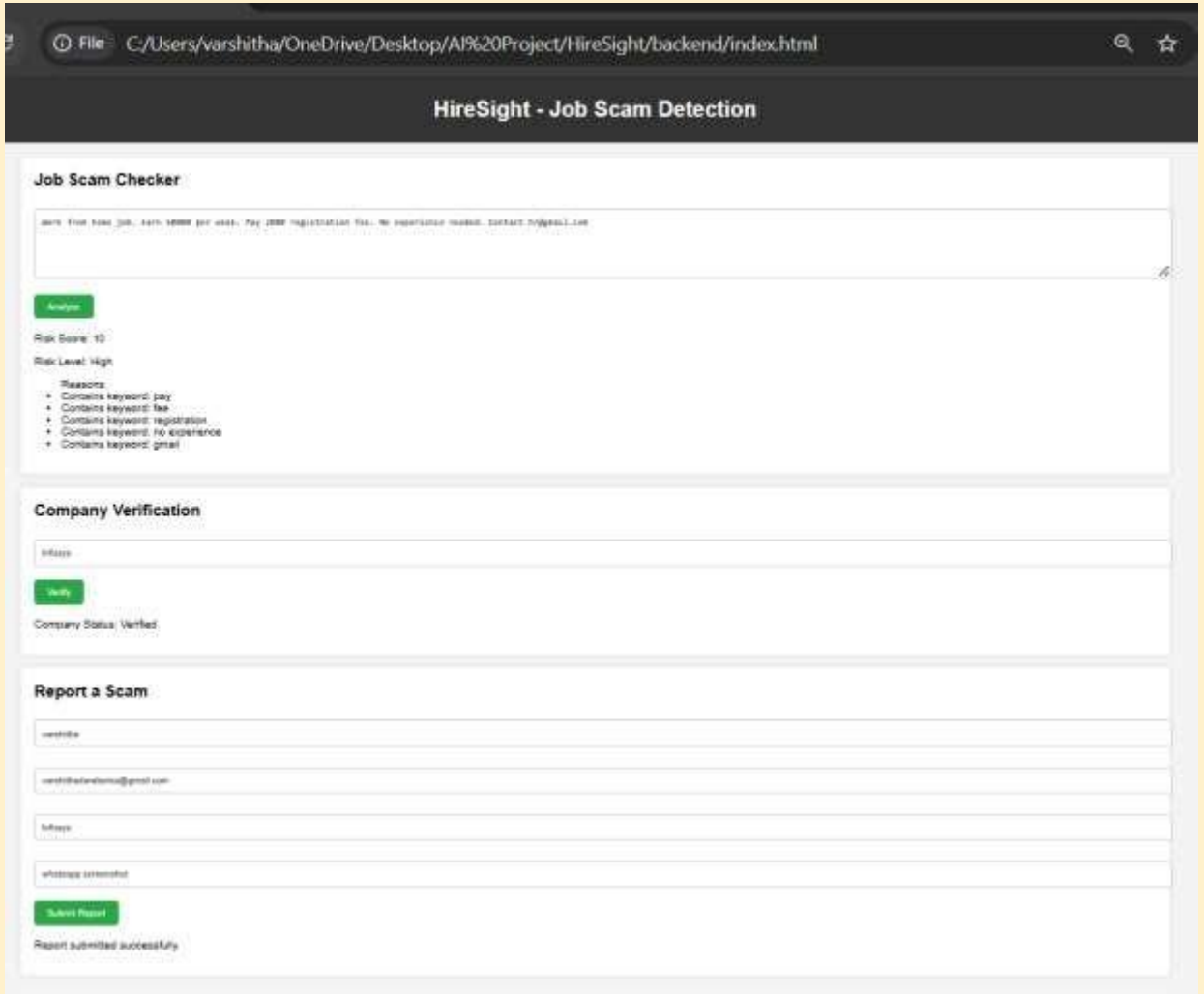
## Prompt Used for Backend Development

- //Generate a Node.js backend using Express and CORS that runs on port 5000. It should include: a GET '/' route returning 'HireSight Backend is Running 🚀', a POST '/analyze' route that takes jobText and uses rule-based logic to calculate scam risk score based on keywords like pay, fee, registration, earn per week, no experience, and gmail/yahoo emails, returning riskScore, riskLevel (Low/Medium/High), and reasons; a GET '/company/:name' route that checks company status from a predefined list (TCS as Verified, Infosys as Verified, FakeJobs Pvt Ltd as Reported as Scam, otherwise Unknown); and a POST '/report' route that validates jobTitle, companyName, description and returns a success message with report details. Use express.json() middleware and proper error handling.

# WORKFLOW (FLOWCHART)



# FRONT-END VIEW



# PROOF OF PROMPTS GENERATING CODES

server.js  index.html ●  README.md

index.html > html

```html
<!--Generate a complete HTML file for a job scam detection web application named Hiresight with clean CSS styling and
a responsive layout.
Include three sections: Job Scam Checker (textarea + analyze button), Company Verification (input + button),
and Report a Scam (multiple inputs + submit button). Use JavaScript with fetch API to connect to a backend
at http://localhost:5000 with endpoints /analyze, /company/:name, and /report. Display risk score, risk level with colors (high, medium, low),
reasons in a list, company status, and report submission response. Include proper error handling and dynamic result display sections.-->

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>HireSight - Job Scam Detection</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 20px;
            background-color: #f4f4f4;
        }
        h1 {
            text-align: center;
            color: #333;
        }
        .container {
            max-width: 800px;
            margin: 0 auto;
            background-color: #fff;
            padding: 20px;
            border-radius: 8px;
            box-shadow: 0 0 10px rgba(0, 0, 0, 0.1);
        }
        .section {
            margin-bottom: 30px;
        }
        .section h2 {
            color: #555;
```

index.html > html > body

```html
<html lang="en">
<head>
    <style>
        .risk-low {
            color: green;
        }
    </style>
</head>
<body>
    <h1>HireSight - Job Scam Detection</h1>
    <div class="container">
        <div class="section" id="job-scam-checker">
            <h2>Job Scam Checker</h2>
            <textarea id="job-description" rows="5" placeholder="Paste the job description here..."></textarea>
            <button onclick="analyzeJob()">Analyze</button>
            <div class="result" id="job-result"></div>
        </div>
        <div class="section" id="company-verification">
            <h2>Company Verification</h2>
            <input type="text" id="company-name" placeholder="Enter company name...">
            <button onclick="verifyCompany()">Verify</button>
            <div class="result" id="company-result"></div>
        </div>
        <div class="section" id="report-scam">
            <h2>Report a Scam</h2>
            <input type="text" id="report-name" placeholder="Your Name">
            <input type="email" id="report-email" placeholder="Your Email">
            <input type="text" id="report-company" placeholder="Company Name">
            <textarea id="report-description" rows="4" placeholder="Description of the scam..."></textarea>
            <button onclick="submitReport()">Submit Report</button>
            <div class="result" id="report-result"></div>
        </div>
    </div>

    <script src="app.js"></script>
```

index.html > html > body

```html
<html lang="en">
<body>
    <div class="container">
        <div class="section" id="report-scam">
            <div class="result" id="report-result"></div>
        </div>
    </div>
    <script>
        async function analyzeJob() {
            const description = document.getElementById('job-description').value;
            try {
                const response = await fetch('http://localhost:5000/analyze', {
                    method: 'POST',
                    headers: { 'Content-Type': 'application/json' },
                    body: JSON.stringify({ description })
                });
                const data = await response.json();
                displayJobResult(data);
            } catch (error) {
                document.getElementById('job-result').innerText = 'Error analyzing job description.';
            }
        }

        function displayJobResult(data) {
            const { riskScore, riskLevel, reasons } = data;
            let riskClass = '';
            if (riskLevel === 'High') riskClass = 'high-risk';
            else if (riskLevel === 'Medium') riskClass = 'medium-risk';
            else riskClass = 'low-risk';

            document.getElementById('job-result').innerHTML = `
                <p>Risk Score: ${riskScore}</p>
                <p class="${riskClass}">Risk Level: ${riskLevel}</p>
                <ul>${reasons.map(reason => `<li>${reason}</li>`).join('')}</ul>
            `;
        }
```

```cpp
    }

    async function verifyCompany() {
        const companyName = document.getElementById('company-name').value;
        try {
            const response = await fetch(`http://localhost:5000/company/${companyName}`);
            const data = await response.json();
            document.getElementById('company-result').innerText = `Company Status: ${data.status}`;
        } catch (error) {
            document.getElementById('company-result').innerText = 'Error verifying company.';
        }
    }

    async function reportScam() {
        const name = document.getElementById('report-name').value;
        const email = document.getElementById('report-email').value;
        const company = document.getElementById('report-company').value;
        const description = document.getElementById('report-description').value;

        try {
            const response = await fetch('http://localhost:5000/report', {
                method: 'POST',
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify({ name, email, company, description })
            });
            const data = await response.json();
            document.getElementById('report
```

server.js ● index.html README.md

server.js > ...

```
//Generate a Node.js backend using Express and CORS that runs on port 5000.
// It should include: a GET '/' route returning 'HireSight Backend is Running 🚀',
// a POST '/analyze' route that takes jobText and uses rule-based logic to calculate scam risk score
// based on keywords like pay, fee, registration, earn per week, no experience, and gmail/yahoo emails, returning riskScore, riskLevel (Low/Mediu
// and reasons; a GET '/company/:name' route that checks company status from a predefined list (TCS as Verified, Infosys as Verified, FakeJobs Pv
// and a POST '/report' route that validates jobTitle, companyName, description and returns a success message with report details. Use express.js

const express = require('express');
const cors = require('cors');

const app = express();
const PORT = 5000;

app.use(cors());
app.use(express.json());

// GET '/' route
app.get('/', (req, res) => {
    res.send('HireSight Backend is Running 🚀');
});
// POST '/analyze' route
app.post('/analyze', (req, res) => {
    const { jobText } = req.body;
    if (!jobText) {
        return res.status(400).json({ error: 'jobText is required' });
    }
```

server.js 1 ● index.html README.md

server.js > ...

```
app.post('/analyze', (req, res) => {
    let riskScore = 0;
    const reasons = [];
    const keywords = ['pay', 'fee', 'registration', 'earn per week', 'no experience', 'gmail', 'yahoo'];
    keywords.forEach(keyword => {
        if (jobText.toLowerCase().includes(keyword)) {
            riskScore += 20;
            reasons.push(`Contains keyword: ${keyword}`);
        }
    });
    let riskLevel = 'Low';
    if (riskScore >= 60) {
        riskLevel = 'High';
    } else if (riskScore >= 40) {
        riskLevel = 'Medium'
    }                    (property) riskLevel: string
    res.json({ riskScore, riskLevel, reasons });
}
);
// GET '/company/:name' route
app.get('/company/:name', (req, res) => {
    const companyName = req.params.name.toLowerCase();
    const companyStatus = {
        'tcs': 'Verified',
        'infosys': 'Verified',
        'fakejobs pvt ltd': 'Reported as Scam'
    };
    const status = companyStatus[companyName] || 'Unknown';
    res.json({ companyName: req.params.name, status });
});
// POST '/report' route
app.post('/report', (req, res) => {
    const { jobTitle, companyName, description } = req.body;
    if (!jobTitle || !companyName || !description) {
        return res.status(400).json({ error: 'jobTitle, companyName, and description are required' });
```

server.js > ...

```javascript
app.get('/company/:name', (req, res) => {
    const companyStatus = {
        'Takejobs pvt ltd': 'Reported as scam'
    };
    const status = companyStatus[companyName] || 'Unknown';
    res.json({ companyName: req.params.name, status });
});
// POST '/report' route
app.post('/report', (req, res) => {
    const { jobTitle, companyName, description } = req.body;
    if (!jobTitle || !companyName || !description) {
        return res.status(400).json({ error: 'jobTitle, companyName, and description are required' });
    }
    const reportDetails = {
        jobTitle,
        companyName,
        description,
        reportedAt: new Date()
    };
    res.json({ message: 'Report submitted successfully', reportDetails });
} );
// Error handling middleware
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).json({ error: 'Something went wrong!' });
});
// Start the server
app.listen(PORT, () => {
    console.log(`Server is running on port ${PORT}`);
});
```

# REFERENCES


- Node.js Official Documentation
- Express.js Documentation
- MDN Web Docs (HTML, CSS, JavaScript)
- Stack Overflow (for debugging errors)
- ChatGPT (for development guidance and code generation)